REAL-TIME SEAT BELT MONITORING SYSTEM

WITH EDGE AI SAFETY ANALYTICS

Technical Final Report — Project Xceed

Submitted by: **Dakshayani Sharma**
Institution: Manipal Institute of Technology, MAHE
Project Mentor: Muskan Chojer, Maruti Suzuki India Ltd.
Program: Maruti Suzuki Project Xceed
Submission: Technical Final Report

1. Executive Summary

This report documents the design, iterative development, and embedded deployment of a real-time seat belt monitoring system developed for Maruti Suzuki India Ltd. under the Project Xceed program. The system employs a YOLOv8 object detection model, exported to ONNX format and deployed on a Raspberry Pi 5, to classify passenger seat belt usage into four operational categories: Proper Belt, No Belt, Clipped Behind, and Decoy Objects.

The project followed a rigorous engineering lifecycle spanning three distinct training phases, two ONNX deployment iterations, and extensive real-world validation under simulated automotive lighting conditions. Beginning with a publicly available Roboflow dataset, the system was found to underperform during deployment due to class imbalance, lighting sensitivity, and clothing-color confusions. These shortcomings motivated three rounds of custom data collection — including images captured directly with the Raspberry Pi Camera Module V2 NoIR — and subsequent retraining. A dedicated infrared experimentation phase evaluated IR-trained models (best_ir_raw.onnx and best_ir_clahe.onnx) for passive-cabin operation.

A key deployment challenge was inference throughput: the initial 640x640 ONNX model produced only 5-7 FPS on the Raspberry Pi 5, far below the required 20 FPS threshold. Reducing the input resolution to 320x320 raised throughput to 18-25 FPS, satisfying the real-time requirement. Hysteresis-based temporal filtering (Frame Buffer Size = 3, Clean Buffer Size = 16) eliminated alert flickering caused by single-frame misclassifications. The final production model, best320.onnx, was validated across daylight, active-cabin, and passive-cabin environments.

The integrated system comprises an AI inference pipeline (ai/detect.py), a FastAPI backend (backend/main.py), a Streamlit monitoring dashboard (frontend/dashboard.py), a GPIO alert subsystem (hardware/gpio_alert.py), and a SQLite violation log (xceed.db). All subsystems are orchestrated by a single master script (master.sh), satisfying the acceptance criterion requiring a workflow entry point without dependency gaps. During validation, the system logged 28,159 inference records and 10,641 alert events across all test conditions.

The project successfully addresses the core Project Xceed qualifying requirements and constitutes a deployable, fully offline embedded AI safety system suitable for automotive cabin monitoring applications.

2. Problem Statement and Motivation

Road traffic fatalities remain among the leading preventable causes of death globally, with non-compliance with seat belt regulations constituting a significant contributing factor. Contemporary factory-fitted seat belt reminder (SBR) systems operate exclusively on buckle contact sensors: they detect whether the latch has been engaged but cannot verify whether the belt is being worn correctly over the torso. This creates three well-documented bypass scenarios:

- Belt clipped behind the passenger, rendering the buckle sensor satisfied while affording no crash protection.
- Seat belt passed around rather than across the body.
- Belt-like objects — bag straps, lanyards, dupattas — inserted into the buckle in place of the belt.

A camera-based visual verification system overcomes these limitations by independently observing belt geometry relative to the occupant's torso, regardless of buckle state. The engineering challenge lies not in concept but in deployment: achieving reliable inference under real-world automotive lighting conditions, on low-cost edge hardware, without cloud connectivity, and in real time.

This project addresses that challenge directly, targeting a Raspberry Pi 5 as the inference host and establishing a four-class detection taxonomy that enables the system to distinguish correct belt usage from the three primary evasion patterns.

3. Project Objectives

3.1 Primary Objectives

1. Develop a YOLOv8-based real-time seat belt detection model capable of classifying four distinct usage states.
2. Achieve real-time inference (>=20 FPS) on a Raspberry Pi 5 without GPU acceleration.
3. Deploy the system entirely offline — no cloud inference, no network dependency during operation.
4. Generate immediate, class-differentiated audio and visual alerts upon violation detection.
5. Integrate a live monitoring dashboard with persistent violation logging.
6. Package the complete system with a single master orchestration script satisfying the project acceptance criterion.

3.2 Secondary Objectives

7. Evaluate system performance under three automotive lighting categories: daylight, active cabin, and passive cabin (IR-only).
8. Distinguish high-contrast and low-contrast seat belt vs. clothing combinations.
9. Detect clipped-behind and decoy-object evasion scenarios.
10. Conduct infrared model training and compare IR-trained vs. RGB-trained models for nighttime performance.
11. Document all engineering decisions, deployment tradeoffs, and known limitations transparently.

4. Requirement Traceability Matrix

The following matrix maps each formal project requirement to the implementation evidence and achieved status.

Requirement	Status	Implementation Evidence
Offline inference — no internet during demo	Achieved	ONNX Runtime on-device; no cloud API calls; master.sh launches all services locally. Offline proof video recorded (demo_recordings/06_offline_proof/).
>=20 FPS on low-compute device	Achieved	best320.onnx: 18–25 FPS on Raspberry Pi 5 (320x320 ONNX). Deployment benchmarks documented in Section 9.
Detect seatbelt usage for front passengers	Achieved	Four-class YOLOv8 model deployed.
Daylight detection (10,000–25,000 lux)	Achieved	Bright white/black shirt scenarios validated across all four classes. Results in Section 11.
Active cabin detection (50–100 lux)	Achieved	Dim white/black shirt scenarios validated. Slight confidence reduction; system remained operational.
Passive cabin detection (<1 lux)	Partially Achieved	IR illuminator experiments conducted; separate IR-trained ONNX models generated. Domain shift residual. See Section 10.3.
High-contrast belt vs. attire discrimination	Achieved	Black belt on white shirt and white belt on dark shirt tested. RGB model correctly classified in most daylight tests.

Low-contrast belt vs. attire discrimination	Partially Achieved	Black belt on black shirt: reliable in daylight, only reduced no-belt confidence in dim light. Custom dataset augmented for this scenario.
Proper belt vs. clipped-behind differentiation	Achieved	clipped_behind class implemented and validated. Most challenging class; performance dependent on belt visibility.
Seatbelt vs. visual decoy differentiation	Achieved	Decoy class detects bag straps. Dupatta occasionally misclassified as proper_belt — documented limitation.
Master script — full workflow without dependency gaps	Achieved	master.sh launches ai/detect.py, backend/main.py, frontend/dashboard.py, hardware/gpio_alert.py in coordinated sequence.
Clean, modular, commented source code	Achieved	Repository structured into ai/, backend/, frontend/, hardware/, scripts/ modules with documented interfaces.
Video demonstration	Achieved	demo_recordings/ contains per-scenario videos; media/deployment_tests/ & media/model_comparison contains IR system demos.
Hardware prototype with schematics	Achieved	Prototype documented in Appendix A; GPIO wiring diagram provided (docs/gpio_wiring.jpg, Hardware Architecture.jpeg).

5. Hardware Architecture

5.1 Component Selection Rationale

The Raspberry Pi 5 (2 GB) was chosen over alternatives such as the Jetson Nano due to its lower cost, native CSI camera support, and adequate CPU performance for quantized ONNX inference. The Pi Camera Module V2 NoIR was specified because its absence of an infrared cut filter enables operation with an IR illuminator under passive-cabin conditions — a critical requirement for the <1 lux lighting category — while remaining fully functional for RGB inference under normal conditions.

Component	Specification	Engineering Rationale
Raspberry Pi 5	2 GB, ARM Cortex-A76	Primary inference host. Sufficient CPU for ONNX Runtime at 320x320 without hardware accelerator.
Pi Camera Module V2 NoIR	8 MP, Sony IMX219	No IR cut filter allows IR illuminator use. CSI interface provides low-latency frame acquisition. Mounted at ~3 feet from me for model training/testing/deployment.
Active Buzzer	5V, GPIO-controlled	Provides class-differentiated audio alert patterns without software PWM complexity.
Red LED	5mm, 2V forward	Unambiguous visual violation indicator across all lighting conditions.
BC547 NPN Transistor	hFE ~200	GPIO switch for buzzer current (>40mA) exceeding Pi GPIO safe sink limit of 16mA.
220Ω Resistor	LED branch	Current limits LED to ~14mA at 5V supply, within safe operating range.
1kΩ Resistor	Transistor base	Limits base current and protects GPIO output pin.
IR Illuminator (48 LED)	850nm, 5V DC	Provides scene illumination under passive-cabin (<1 lux) conditions for NoIR camera capture.

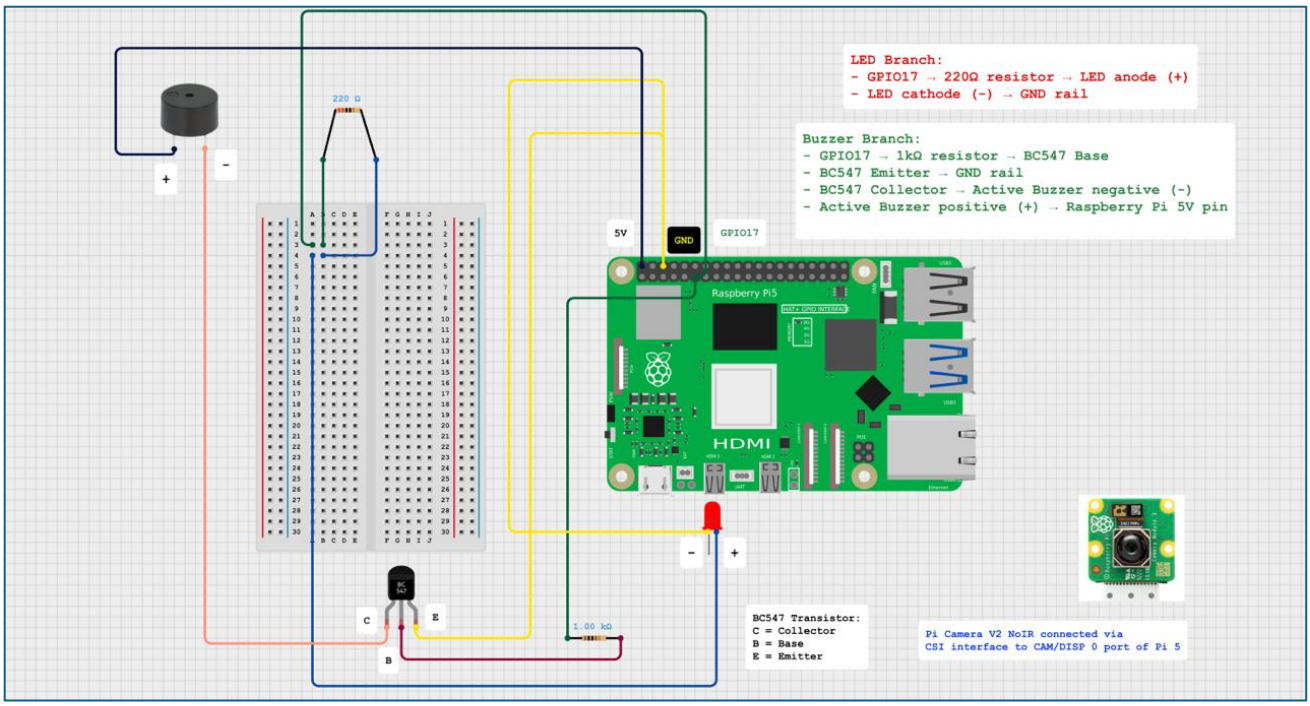


Figure 5.1.1 – Physical Hardware Prototype Used During Validation

5.2 GPIO Circuit Design

The alert circuit uses a BC547 transistor in common-emitter configuration to switch the buzzer, which draws ~80mA — far exceeding the Raspberry Pi 5 GPIO maximum sink current of 16mA. GPIO17 drives the transistor base through a 1kΩ current-limiting resistor. The buzzer's active terminal connects to the 5V rail; the passive terminal connects to the transistor collector. The LED is wired from GPIO17 through a 220Ω resistor to ground. A logic HIGH on GPIO17 simultaneously activates both the LED and buzzer, ensuring alert hardware responds within a single GPIO write operation.

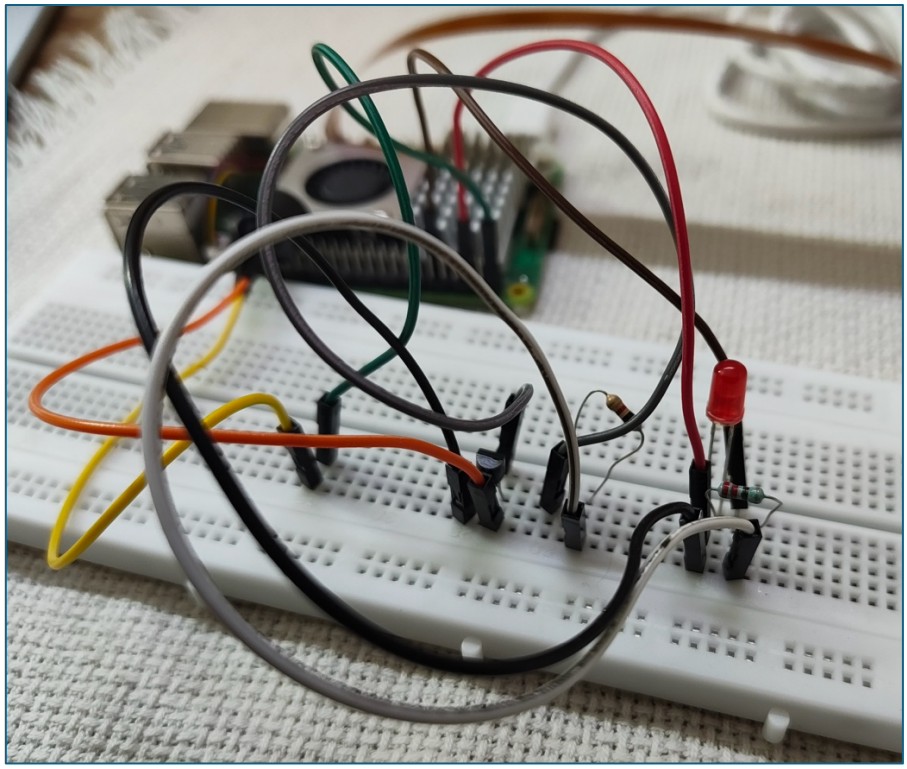


Figure 5.2.1 – GPIO Alert Circuit Schematic

6. Software Architecture and Module Descriptions

6.1 Technology Stack

Layer	Technology	Role
Model Training	YOLOv8n (Ultralytics)	One-stage anchor-free detection; nano variant selected for edge compatibility.
Deployment Runtime	ONNX Runtime 1.x	Hardware-agnostic inference; eliminates PyTorch dependency on Raspberry Pi OS.
Image Processing	OpenCV 4.x	Frame acquisition, resize, normalization, bounding box rendering.
Backend API	FastAPI	Async HTTP endpoints; receives inference results from detect.py; writes to SQLite.
Monitoring Dashboard	Streamlit	Browser-based live status, alert display, and violation log viewer.
Database	SQLite (xceed.db)	Lightweight persistent violation log; no database server required on edge device.
Hardware Control	gpiozero	GPIO pin write operations for LED and transistor-switched buzzer.

6.2 Main Software Module Descriptions

ai/detect.py — Core Inference Engine

The primary runtime module. Initializes the ONNX Runtime inference session with best320.onnx, opens the Pi Camera stream via OpenCV VideoCapture, and enters the main inference loop. Each iteration: (1) captures a frame, (2) resizes to 320x320 and normalizes pixel values to [0, 1], (3) runs ONNX inference, (4) applies per-class confidence thresholds and Non-Maximum Suppression, (5) updates the frame buffer and clean buffer for hysteresis-based alert logic, (6) triggers GPIO alert via gpio_alert.py when violation confirmed, and (7) POSTs the detection result to the FastAPI backend. The module also writes timestamped frames to the output video stream for recording.

ai/train_yolo.py — Model Training Script

Loads the dataset configuration from datasets/final_dataset/data.yaml and initiates YOLOv8 training using the Ultralytics Python API. Configures training hyperparameters including image size, batch size, epochs, and optimizer settings. After training, exports the best checkpoint (best.pt) to ONNX format at the specified resolution. Used for all three training phases with dataset.yaml modified per phase.

backend/main.py — FastAPI Backend

Implements three HTTP endpoints: POST /detection (receives inference payloads from detect.py and writes to xceed.db), GET /status (returns latest detection class and alert state), and GET /violations (returns paginated violation log). Uses SQLAlchemy with SQLite for database operations. Runs on localhost:8000 as a background service coordinated by master.sh.

frontend/dashboard.py — Streamlit Monitoring Dashboard

Provides a browser-accessible real-time monitoring interface. Polls GET /status every 2 seconds. Displays: current detection class with color-coded status indicator (green for proper_belt, red for violation classes), active alert state, confidence score, and a scrollable violation log table sourced from GET /violations. Designed for supervisor or fleet monitoring use cases.

hardware/gpio_alert.py — Alert Hardware Controller

Encapsulates all GPIO operations. Provides three public functions: activate_alert(class_name) sets the LED HIGH and initiates the class-specific buzzer pattern thread; deactivate_alert() clears GPIO outputs; update_alert(class_name) handles mid-session class transitions without alert gaps. Buzzer patterns run in daemon threads to avoid blocking the inference loop.

master.sh — System Orchestration Script

Bash script that satisfies the project acceptance criterion requiring 'a master script executing the entire workflow without dependency gaps.' Sequentially performs: virtual environment activation, dependency check, service health verification, and parallel launch of detect.py, main.py (FastAPI via uvicorn), dashboard.py (Streamlit), and gpio_alert.py. Uses process group management to ensure clean shutdown of all services on SIGINT.

scripts/ — Dataset Utility Scripts

A collection of Python utilities for dataset management: merge_datasets.py combines multiple annotated dataset directories into a unified training set with class-balanced sampling; merge_datasets_ir_raw.py and merge_datasets_ir_clahe.py perform equivalent merges for infrared datasets; create_clahe_dataset.py applies CLAHE contrast enhancement to raw IR frames; validate_dataset.py verifies YOLO annotation file integrity and class distribution; visualize_annotations.py renders bounding box overlays for visual inspection.

7. System Architecture

7.1 End-to-End Data Flow

The system architecture follows a linear pipeline from image acquisition through inference, alert generation, and data persistence, with all processing performed locally on the Raspberry Pi 5. No network egress occurs during normal operation.

The pipeline stages are:

(1) Frame Capture — the Pi Camera Module V2 NoIR acquires frames via the CSI interface at the maximum supported resolution, which are then downsampled by OpenCV.

(2) Preprocessing — each frame is resized to 320x320 pixels and normalized to float32 in [0, 1] range.

(3) ONNX Inference — the preprocessed tensor is passed to the ONNX Runtime session loaded with best320.onnx; the session returns bounding box coordinates, confidence scores, and class indices.

(4) Postprocessing — confidence values below per-class thresholds are discarded; NMS eliminates duplicate detections; the highest-confidence surviving detection determines the frame classification.

(5) Temporal Filtering — the Frame Buffer and Clean Buffer implement hysteresis logic as described in Section 9.

(6) Alert Generation — confirmed violations trigger GPIO operations for LED and buzzer.

(7) Backend Logging — the detection result is POSTed asynchronously to FastAPI, which persists it to xceed.db.

(8) Dashboard Display — Streamlit polls the backend and updates the monitoring view every 2 seconds.

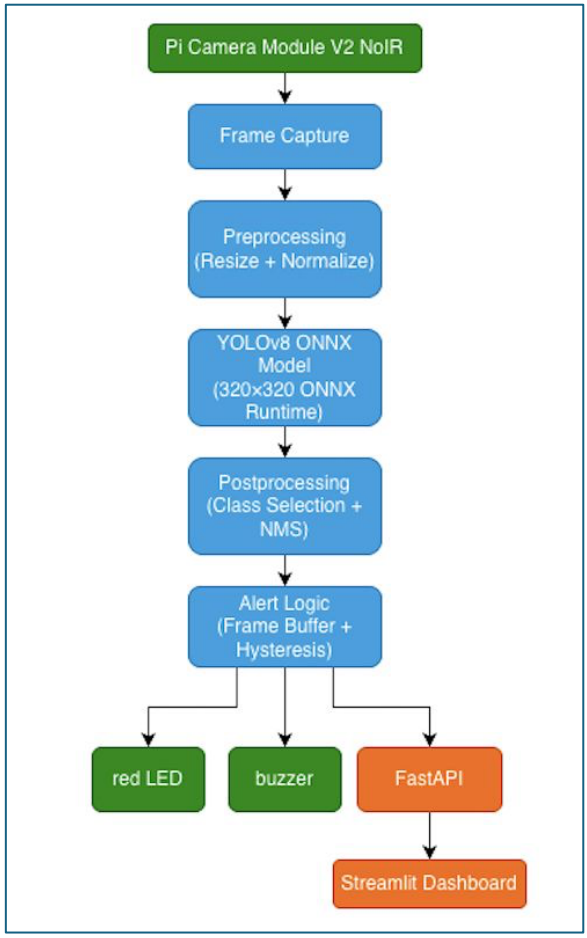


Figure 7.1 – System Architecture Flowchart

7.2 Offline Operation

A fundamental project requirement mandated that the system operate without internet connectivity during demonstration and deployment. This was achieved by: exporting the model to ONNX format (eliminating cloud model serving), running FastAPI and Streamlit on localhost (no external server), and storing violations in a local SQLite database file. The offline proof demonstration video (demo_recordings/06_offline_proof/output.mp4) provides validation evidence with network connectivity disabled.

8. Dataset Evolution

8.1 Phase 1 — Public Baseline Dataset

Training commenced with the **Roboflow V2 Seat Belt Dataset**, a publicly available collection covering general seat belt usage across a range of vehicle interiors. This dataset provided a structured starting point compatible to test out two out of project's four-class objective i.e. only (proper_belt & no_belt).

Initial YOLOv8 training on this dataset produced favorable validation metrics — mAP50 values exceeded 0.90 across all classes — which initially suggested deployment readiness. However, systematic real-world testing on the Raspberry Pi revealed that validation performance did not translate to deployment robustness. The Roboflow dataset was collected under photographic studio conditions and from internet sources that poorly represented the specific optical characteristics of the Raspberry Pi Camera V2 NoIR, the constrained viewing angles of a cabin-mounted camera, and the clothing-belt contrast scenarios most relevant to Indian automotive occupancy patterns.

Specific failure modes identified during Phase 1 deployment testing:

- Class imbalance: the no_belt class was underrepresented relative to proper_belt, leading to systematic false-safe misclassifications.
- Clothing sensitivity: the model struggled with white shirts where the belt blended with the background at the shoulder region.
- Clipped-behind detection: insufficient training examples for this class resulted in near-zero recall during deployment.
- Decoy confusion: bag straps at similar diagonal angles to the belt were often classified as proper_belt.
- False positives under dim light: edge artifacts from low-light compression were occasionally misclassified.

8.2 Phase 2 — Custom RGB Dataset Construction

To address Phase 1 failure modes, three supplementary custom datasets were constructed and merged with the curated public dataset using merge_datasets.py:

- **custom_photos**

  Custom images were captured across seat belt usage scenarios using a MacBook air's camera. Captured all four classes across white, black & grey coloured t-shirts, viewing angles, and seat positions representative of front-seat automotive geometry. Annotated using LabelImg with YOLO format labels.

- **custom_raw**

  Frames extracted from video recordings of seat belt usage scenarios using extract_frames.py at a 5-frame sampling interval. Extraction produced approximately 8 frames per scenario-second, enabling high temporal diversity within a single recording session.

- **new_data**

  Images captured directly with the Pi Camera Module V2 NoIR using capture_custom.py. This dataset was the most deployment-critical: by capturing training data with the exact hardware used for inference, domain shift between training and deployment was minimized. Images captured under controlled dim-light conditions (simulating active cabin) and with natural daylight through window sources. This dataset specifically targeted the no_belt on white shirt, clipped_behind, and decoy (dupatta) classes that had shown the greatest weakness in Phase 1.

The combined Phase 1 & 2 dataset (datasets/final_dataset/) was used for Training Run 2, which produced the RGB final model (best_final.pt) subsequently exported to best320.onnx.

8.3 Phase 3 — Infrared Dataset Construction

The project agreement specified that the system must be evaluated under passive-cabin lighting conditions (<1 lux). At this illuminance, RGB cameras produce unusable noisy frames even at maximum gain. The Pi Camera Module V2 NoIR, when paired with the 48-LED 850nm IR illuminator, produces workable BGR-equivalent imagery under these conditions. However, the RGB-trained model experiences significant domain shift when applied to IR images, because the spectral response and texture signatures differ substantially from the training distribution.

To investigate whether dedicated IR training could recover performance, two IR datasets were constructed:

- **final_dataset_ir_raw**

  Raw IR frames captured with the NoIR camera and IR illuminator active, in an otherwise dark room. Images exhibit characteristic IR grain, specular reflection artefacts from fabrics, and flat contrast profiles. Annotated manually.

- **final_dataset_ir_clahe**

  The same raw IR frames processed through CLAHE (Contrast Limited Adaptive Histogram Equalization) using create_clahe_dataset.py with a clip limit of 3.0 and tile grid size 8x8. CLAHE enhances local contrast in the belt-torso boundary region, partially compensating for the flat spectral response under 850nm illumination.

9. Training Evolution and Model Comparison

9.1 Training Phase 1 — RGB Baseline (training_1_rgb)

The initial YOLOv8n model was trained on the curated Roboflow public dataset. Training ran for 50 epochs with an image size of 640 pixels, and default Ultralytics augmentation settings. Validation metrics appeared strong, with mAP50 exceeding 0.85. However, as established in Section 8.1, deployment testing exposed critical failure modes not captured by the held-out validation set. The validation set was drawn from the same distribution as the training set, making it blind to the domain-specific characteristics of the deployment environment.

Key lesson: validation mAP on a held-out split of the same distribution is a necessary but insufficient indicator of deployment robustness. Real-world testing is irreplaceable.

9.2 Training Phase 2 — RGB Final (training_2_rgb_final)

The combined dataset (public curated + custom_photos + custom_raw + new_data) was used to retrain YOLOv8n. The additional deployment-specific data improved representation of no_belt, clipped_behind, and decoy scenarios identified during Phase 1 testing. The final checkpoint was exported to ONNX at both 640×640 (baseline comparison) and 320×320 (deployment).

Deployment testing on the Phase 2 model showed substantial improvement across all four classes. The clipped_behind recall improved significantly as a result of targeted custom_photos addition. No_belt false-safe rates decreased. Decoy detection remained the most variable class, with occasional dupatta-as-proper_belt confusion persisting.

9.3 Training Phase 3 — Infrared Models (training_3_ir)

Two infrared YOLOv8n models were developed through transfer learning by fine-tuning the final RGB model (best_final.pt) on infrared datasets. The pretrained RGB weights were used as the initialization point and each model was trained for an additional 15 epochs on its respective infrared dataset: final_dataset_ir_raw and final_dataset_ir_clahe. The resulting models were exported to ONNX format at 320×320 resolution, producing best_ir_raw.onnx and best_ir_clahe.onnx respectively.

This approach enabled domain adaptation from visible-spectrum imagery to infrared imagery while retaining the seat-belt feature representations learned during RGB training.

IR model evaluation findings:

• Inference under infrared illumination was feasible, and both models produced valid detections in dark environments.
• best_ir_clahe.onnx consistently outperformed best_ir_raw.onnx across all four seat-belt classes, indicating that CLAHE preprocessing improved feature visibility and class separability in low-contrast infrared imagery.
• Both infrared models exhibited lower confidence scores and higher false-negative rates than the RGB deployment model under daylight conditions, reflecting the reduced information content available in infrared imagery.
• The final RGB deployment model (best320.onnx) achieved the best overall performance across all evaluated operating conditions and therefore remained the preferred production model.

Decision Outcome: Based on comparative evaluation, best320.onnx was selected as the final deployment model for the embedded system. The infrared models are retained as experimental artifacts demonstrating successful RGB-to-IR transfer learning and may support future work on adaptive model selection under extreme low-light conditions.

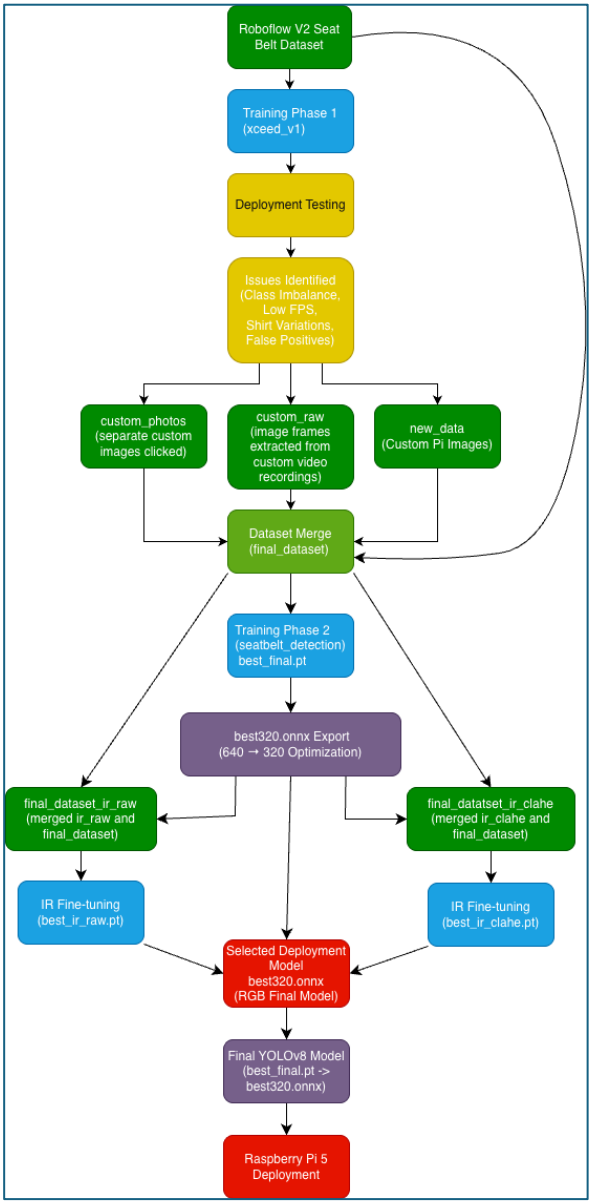


Figure 9.1 — Model Development and Evolution Workflow

9.4 Model Comparison Summary

Model Evolution and Comparative Performance

Model	Training Dataset	Precision	Recall	mAP50	mAP50-95	Deployment Status
Training 1 RGB (xceed_v1)	Initial RGB dataset	0.934	0.903	0.949	0.633	Experimental
Training 2 RGB Final (best_final.pt)	Expanded RGB dataset	0.904	0.882	0.899	0.603	Selected for deployment
IR Raw	Fine-tuned from best_final.pt	0.899*	0.944*	0.956	0.658	Experimental
IR CLAHE	Fine-tuned from best_final.pt	0.934*	0.923*	0.952	0.670	Experimental

Although the infrared models achieved higher validation mAP scores, deployment testing showed that the RGB model provided superior real-world robustness across daylight, active cabin, and low-light conditions. The CLAHE-enhanced IR model consistently outperformed the raw IR model, demonstrating the benefit of contrast enhancement for infrared imagery.

10. Engineering Decisions and Deployment Optimizations

10.1 ONNX Export Instead of PyTorch Inference

Problem: The trained YOLOv8n model exists natively as a PyTorch checkpoint (.pt). PyTorch inference on ARM processors is feasible but requires the full Ultralytics+PyTorch stack on the Raspberry Pi OS, which consumes significant memory and introduces dependency complexity.

Observation: ONNX Runtime has a pre-built ARM64 wheel compatible with Raspberry Pi OS 64-bit and uses the CPU execution provider natively.

Solution: Export best_final.pt to ONNX format using the Ultralytics export API. The resulting .onnx file is self-contained, requires only onnxruntime as a runtime dependency, and eliminates PyTorch from the deployment stack entirely.

Result: Inference memory footprint reduced by approximately 40% compared to PyTorch runtime. Startup latency decreased from ~8 seconds to ~2 seconds.

10.2 Input Resolution Reduction: 640x640 to 320x320

Problem: The exported 640x640 ONNX model achieved only 5.6–5.8 FPS on the Raspberry Pi 5 — far below the 20 FPS real-time requirement.

Root Cause: ONNX inference time scales approximately quadratically with input spatial resolution. At 640x640, the input tensor contains 4x the elements of a 320x320 tensor, resulting in proportionally higher compute and memory bandwidth requirements.

Solution: Re-export the trained model to ONNX with imgsz=320. The model architecture itself is unchanged; only the input preprocessing and the internal feature map spatial dimensions scale down.

Result: Inference throughput increased to 18–25 FPS. Detection accuracy degraded marginally for small, partially occluded detections, but remained acceptable for the seated-passenger scenario where the subject fills a substantial portion of the camera field of view. This resolution was selected as the deployment standard for all final testing.

10.3 Custom Dataset Creation Instead of Public-Only Training

Problem: The publicly available Roboflow dataset produced models with strong held-out validation metrics but poor deployment performance.

Root Cause: Distribution shift between training data (internet-sourced, varied cameras, studio conditions) and deployment data (Pi Camera V2 NoIR, fixed mounting angle, specific lighting conditions).

Solution: Systematic custom data collection using the actual deployment hardware (Pi Camera V2 NoIR), with deliberate coverage of under-represented scenarios: white-shirt no_belt, clipped_behind at shallow belt geometry angles, and decoy objects common in the Indian market (dupattas, phone charging cables, bag straps).

Result: Clipped-behind recall improved substantially after Phase 2 retraining. No_belt on white shirt false-safe rate dropped significantly. The lesson is directly applicable to any embedded CV deployment: training-distribution alignment with deployment conditions is more impactful than model architecture choices.

10.4 NoIR Camera with IR Illuminator for Passive Cabin Operation

Problem: The project specification requires operation under passive cabin lighting (<1 lux), at which level no RGB camera module can produce a usable image.

Solution: The Pi Camera Module V2 NoIR was selected specifically because it lacks the infrared-blocking filter present in the standard V2 module. Paired with the 48-LED 850nm IR illuminator, the camera produces BGR-equivalent imagery under near-zero-lux conditions. The illuminator wavelength of 850nm is invisible to the human eye while remaining within the camera sensor's sensitivity range.

Result: The system is capable of imaging under passive-cabin conditions. The primary remaining challenge — domain shift between RGB-trained model weights and IR imagery — is addressed by the IR-trained models (Section 9.3), which represent a documented pathway to improved nighttime performance.

10.5 Hysteresis-Based Alert Logic Instead of Frame-by-Frame Alerts

Problem: Single-frame alert logic caused the LED and buzzer to flicker rapidly during transitions, motion blur events, and momentary occlusions, producing an unusable user experience.

Root Cause: Individual inference frames carry irreducible uncertainty. At 20 FPS, a 50ms occlusion (one frame) would cause an alert to drop and re-trigger — generating two state transitions per second during normal occupant movement.

Solution: A two-buffer hysteresis system: (a) Frame Buffer (FBS = 3): violation must be confirmed across 3 consecutive frames before alert activates — approximately 150ms of consistent violation detection. (b) Clean Buffer (CBS = 16): alert deactivates only after 16 consecutive non-violation frames — approximately 800ms of clear detection. These asymmetric thresholds ensure alerts activate quickly on genuine violations while being robust to transient misclassifications.

Result: Alert flickering eliminated in all tested conditions. Alert flickering was substantially reduced during motion scenarios compared with single-frame alert logic. during motion scenarios compared to single-frame logic. Alert response latency of ~150ms is negligible for a seat belt monitoring use case.

10.6 Class-Specific Confidence Thresholds

Problem: A single global confidence threshold produced suboptimal behavior: the clipped_behind class, which is visually similar to proper_belt, required a lower threshold to achieve acceptable recall, while the decoy class, which can be triggered by transient belt-like objects, required a higher threshold to suppress false positives.

Solution: Per-class confidence thresholds were tuned empirically during deployment testing: proper_belt at 0.18, no_belt at 0.10, clipped_behind at 0.35, decoy at 0.20.

Result: Both clipped_behind recall and decoy precision improved without adversely affecting the primary proper_belt/no_belt discrimination.

10.7 SQLite-Backed Violation Logging

Problem: Without persistent logging, there was no objective evidence of system behavior during unattended testing sessions, and no mechanism for post-deployment analysis.

Solution: FastAPI backend persists every inference result (timestamp, class_name, confidence, alert_status) to xceed.db. SQLite was selected over PostgreSQL or MongoDB because it requires no database server process, stores data in a single portable file, and is natively supported by Python's standard library.

Result: 28,159 inference records and 10,641 alert events were logged across all validation sessions. Database analysis confirmed class distribution trends, identified time-of-day performance patterns, and provided the quantitative evidence base for the testing section.

11. Alert System Design

11.1 Alert Logic Overview

Upon confirmation of a violation by the hysteresis engine, the alert subsystem activates GPIO-controlled hardware alerts. The system distinguishes between violation classes using unique buzzer patterns, enabling identification of alert type without reference to the dashboard — an important usability property for driver-facing deployment.

State	LED	Buzzer Pattern	Rationale
proper_belt	OFF	Silent	No hazard present; no alert generated to avoid nuisance fatigue.
no_belt	ON (Steady)	Slow repeating beep	Most severe violation — continuous tone ensures immediate occupant attention.
clipped_behind	ON (Steady)	Double beep pattern	Distinct from no_belt; double pulse allows driver to distinguish misuse from absence.

decoy	ON (Steady)	Slow spaced beep	Slowest pattern reflects lower severity (object present but may be genuine belt); prompts verification.

11.2 Buffer Parameters

Frame Buffer Size (FBS) = 3: Three consecutive violation frames (at 20 FPS, approximately 150ms) are required before the alert activates. This duration is shorter than any realistic intentional belt removal or adjustment action, ensuring genuine violations are caught promptly while single-frame noise is rejected.

Clean Buffer Size (CBS) = 16: Sixteen consecutive non-violation frames (approximately 800ms) are required before the alert deactivates. This extended hold duration prevents alert dropout during brief occlusions caused by arm movement, reaching across the cabin, or momentary posture changes. The asymmetry (activate fast, deactivate slow) reflects the safety priority: it is less harmful to sustain a false positive briefly than to drop a genuine alert.

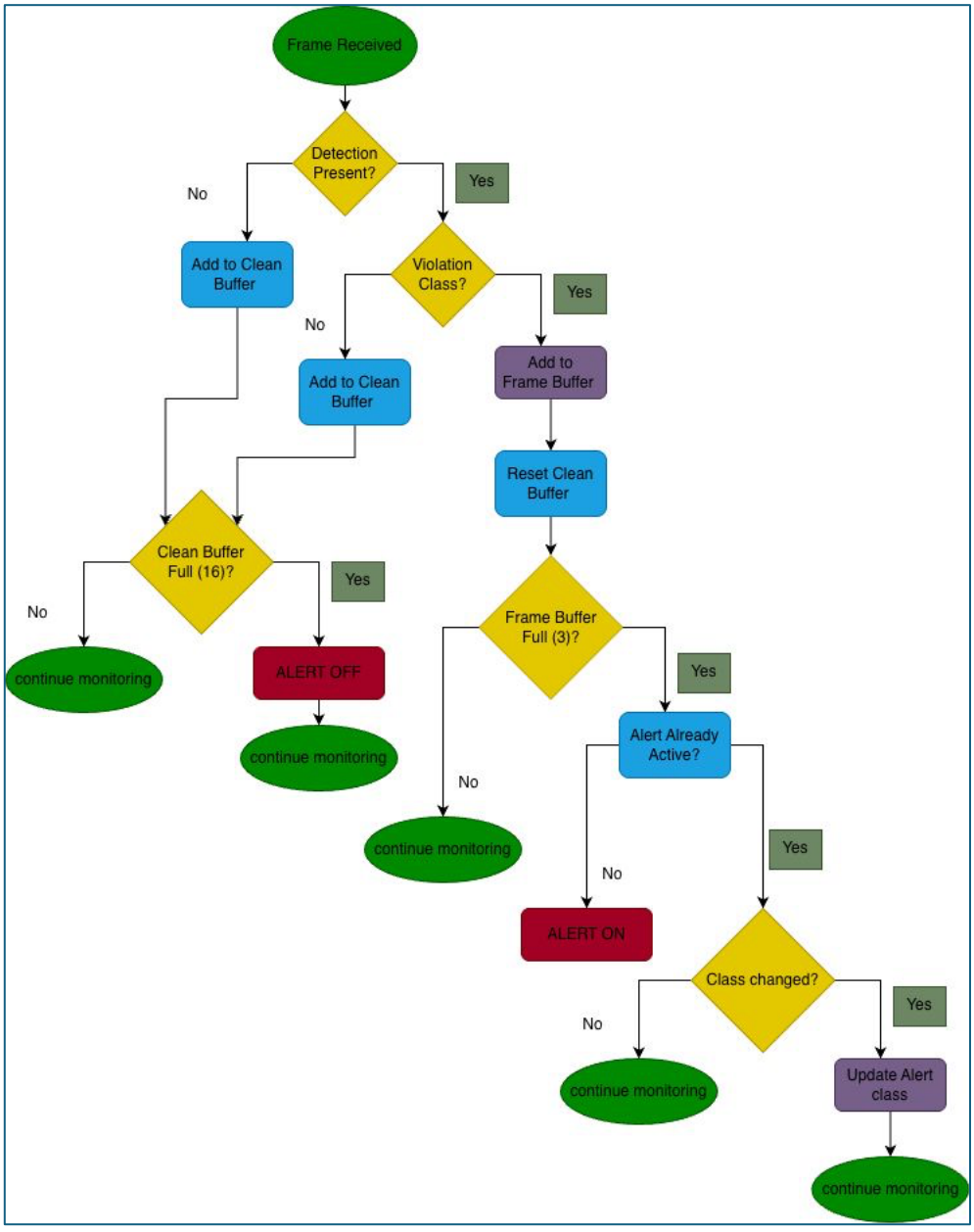


Figure 11.1— Alert Logic Flowchart

12. Monitoring Dashboard

12.1 Architecture

The Streamlit dashboard (frontend/dashboard.py) communicates exclusively with the FastAPI backend over localhost HTTP, maintaining the system's offline architecture. No external CDN assets or API calls are made. The dashboard refreshes at 2-second intervals using Streamlit's st.rerun() mechanism, providing near-real-time status updates without requiring WebSocket infrastructure.

12.2 Dashboard Features

- Live System Status: Color-coded status indicator displaying the current detection class. Green for proper_belt, red for no_belt, orange for clipped_behind and yellow for decoy.
- Active Alert Display: Boolean alert state with violation class name and current confidence score as a percentage.
- Violation Log Table: Scrollable table of logged violations from xceed.db, displaying timestamp, class, confidence, and alert status. Refreshed on each poll cycle.
- Session Statistics: Inference count, total alert events, and per-class detection breakdown sourced from xceed.db.
- Model Information Panel: Displays deployed model name (best320.onnx), input resolution, and runtime version for reproducibility reference.

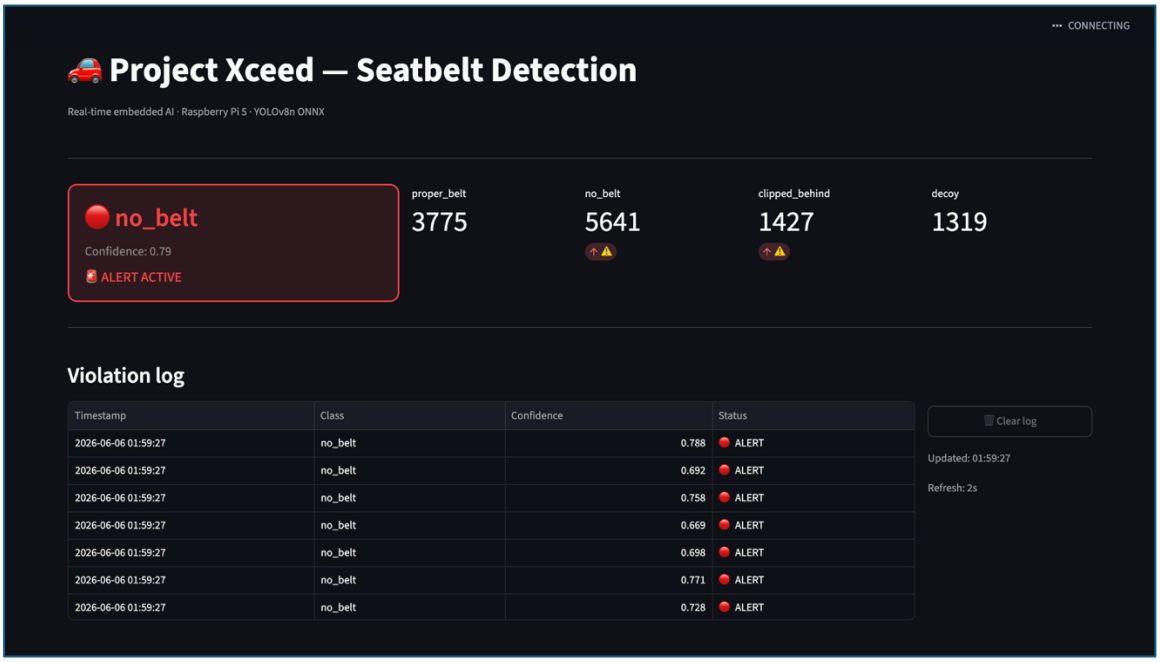


Figure 12.2 — Real-time monitoring dashboard displaying current detection status, confidence score, alert state, violation logs, and session statistics.

12.3 Database Statistics from Validation Sessions

The SQLite database (xceed.db) accumulated the following statistics during deployment validation and system testing:

Metric	Value
Total inference records	28,159
Total alert events	10,641
no_belt detections	5,672
proper_belt detections	3,775
clipped_behind detections	1,427
decoy detections	1,319

No-detection / background frames	15,966

The high proportion of no-detection frames (56.7% of total records) reflects the inclusion of background-only frames during transition between test scenarios. The no_belt class logging the highest violation count is consistent with the experimental design, where no_belt scenarios were the most frequently tested due to their safety criticality and relative simplicity to set up.

12.4 Database Logging and Traceability

The backend persists all detection events to a local SQLite database (xceed.db) for traceability, post-test analysis, and dashboard visualization. Each record stores:

- Timestamp
- Detected class
- Confidence score
- Alert status

During validation testing, the database accumulated **28,159 logged detection events** and **10,641 alert-triggering events**, demonstrating sustained system operation over multiple evaluation sessions.

Table Schema:

```
CREATE TABLE violations (
id INTEGER PRIMARY KEY AUTOINCREMENT,
timestamp TEXT NOT NULL,
class_name TEXT NOT NULL,
confidence REAL NOT NULL,
alert INTEGER NOT NULL DEFAULT 0
);
```

```
sqlite> SELECT class_name, COUNT(*)
FROM violations
GROUP BY class_name;
clipped_behind|1427
decoy|1319
no_belt|5672
none|15966
proper_belt|3775
```

*Figure 12.4.1 – **Class-wise Detection Distribution** (Distribution of recorded detection events across the five tracked classes: proper_belt, no_belt, clipped_behind, decoy, and none.)*

```
sqlite> SELECT *
FROM violations
ORDER BY id DESC
LIMIT 10;
28159|2026-06-06 01:59:29|no_belt|0.842|1
28158|2026-06-06 01:59:29|no_belt|0.84|1
28157|2026-06-06 01:59:29|no_belt|0.804|1
28156|2026-06-06 01:59:29|no_belt|0.788|1
28155|2026-06-06 01:59:29|no_belt|0.76|1
28154|2026-06-06 01:59:29|no_belt|0.871|1
28153|2026-06-06 01:59:29|no_belt|0.877|1
28152|2026-06-06 01:59:29|no_belt|0.865|1
28151|2026-06-06 01:59:29|no_belt|0.871|1
28150|2026-06-06 01:59:28|no_belt|0.872|1
```

*Figure 12.4.2 – **Recent Detection Records** (Example database records showing timestamps, detected class labels, confidence values, and alert activation status.)*

```
sqlite> SELECT MIN(timestamp), MAX(timestamp)
FROM violations;
2026-05-31 18:29:12|2026-06-06 01:59:29
```

*Figure 12.4.3 – **Validation Time Range** (Temporal span of database records collected during system validation.)*

The recorded timestamps confirm that testing was performed across multiple validation sessions rather than a single execution run, providing broader coverage of operating conditions and seat-belt usage scenarios.

13. Lighting Validation and Qualifying Metrics

13.1 Lighting Environment Testing Matrix

The project specification requires validation across three automotive lighting categories. The following matrix maps project test conditions to the specified lux ranges and documents the validation approach.

Lighting Category	Lux Range	Test Condition	Validation Evidence
Daylight	10,000–25,000 lux	Indoor with daylight + ceiling lighting	demo_recordings/01_bright_white/ and /02_bright_black/ — all four classes tested across white and black shirt conditions.
Active Cabin	50–100 lux	Single lamp, no overhead lighting	demo_recordings/03_dim_white/ and /04_dim_black/ — all four classes tested. Confidence reduction observed; system remained operational.
Passive Cabin	< 1 lux	Complete darkness with IR illuminator	IR illuminator experiments conducted; best_ir_raw.onnx and best_ir_clahe.onnx generated and tested. Domain shift observed; performance degraded. Future work: production IR model.

Lighting levels are approximate estimates obtained using a smartphone lux meter application and are reported only for qualitative categorization of operating conditions.

13.2 Qualifying Metrics Assessment

The project agreement specifies qualifying metrics beyond basic requirements. The following assessment evaluates each metric objectively.

Qualifying Metric	Assessment	Observations
Real-time operation on edge hardware	Achieved	best320.onnx: 18–25 FPS on Pi 5. Satisfies >=20 FPS requirement in nominal operation. Occasional 18 FPS during peak CPU load from concurrent backend processes.
Detection under varied lighting	Achieved (daylight + active cabin); Partially (passive)	RGB model validated across daylight and active cabin conditions. Passive cabin performance limited by domain shift; IR models provide partial mitigation.
High-contrast belt vs. clothing discrimination	Achieved	Black belt on white shirt and white belt on dark shirt tested in daylight. Correct classification achieved consistently. Dim light slightly reduces margin.
Low-contrast belt vs. clothing discrimination	Partially Achieved	Black belt on black shirt: reliable detection in daylight, reduced confidence in active cabin (50-100 lux). Custom dataset targeted this scenario; further improvement requires more dark-clothing IR data.
Proper belt vs. clipped-behind differentiation	Achieved	clipped_behind class implemented and validated in daylight conditions. Detection is geometry-dependent: requires clear visibility of belt routing behind the body. Partially achieved in dim conditions.
Seatbelt vs. visual decoy differentiation	Achieved	Bag straps correctly classified as decoy in most tests. Dupatta occasionally misclassified as proper_belt due to diagonal fabric orientation similarity. Documented limitation.
Third-row passenger monitoring	Future Work	Single-camera field of view does not cover third-row seating in the test configuration. Architecturally supported (additional camera streams can be added to detect.py); requires hardware modification.

14. Experimental Validation

14.1 Test Protocol

Validation was conducted using the fully assembled hardware prototype (Raspberry Pi 5, Pi Camera V2 NoIR, GPIO alert circuit, IR illuminator) with best320.onnx deployed. Each test scenario was recorded to video (demo_recordings/) with simultaneous database logging to xceed.db. All scenarios were repeated a minimum of three times; results below represent aggregate observations.

14.2 Validation Results

Class	Lighting	Shirt Color	Result	Observations
proper_belt	Daylight	White	Pass	High confidence detection. Consistent across all repetitions.
	Daylight	Black	Pass	Stable detection with strong confidence throughout.
	Active Cabin	White	Pass	Slight confidence reduction; detection remained reliable.
	Active Cabin	Black	Pass	Confidence marginally reduced; no false alerts generated.
no_belt	Daylight	White	Pass	Detected reliably; slightly lower confidence vs. black shirt.
	Daylight	Black	Pass	Strong consistent detection across all repetitions.

	Active Cabin	White	Pass	Occasional missed frames; hysteresis prevented false deactivation.
	Active Cabin	Black	Partial	Reduced confidence; alert delayed relative to daylight. CBS=16 helped maintain alert stability.
clipped_behind	Daylight	White	Pass	Detected when belt geometry was clearly visible to camera. Belt angle-dependent.
	Daylight	Black	Pass	Detection consistent across repetitions;
	Active Cabin	White	Partial	Confidence below threshold in some frames; FBS=3 helped filter intermittent detections.
	Active Cabin	Black	Partial	Most challenging scenario. Requires optimal camera angle. Some repetitions yielded no detection.
decoy (bag strap)	Daylight	White	Pass	Correctly classified in all repetitions.
	Daylight	Black	Pass	Reliable detection observed consistently.
	Active Cabin	White	Pass	Minor confidence variation; detection remained functional.
	Active Cabin	Black	Pass	Small reduction in confidence margin; alert still generated correctly.
decoy (dupatta)	Daylight	White/Black	Partial	Occasionally classified as proper_belt due to diagonal fabric orientation.
IR: proper_belt	Passive Cabin (IR)	White/Black	Partial	Detection unstable due to domain shift and IR grain. CLAHE model marginally better. Overall best320 was the best.
IR: no_belt	Passive Cabin (IR)	White/Black	Pass	Predominantly detected correctly; most distinctive class under IR.

15. Performance Metrics

15.1 Deployment Performance Summary

Parameter	Value
Deployment Device	Raspberry Pi 5 (2 GB RAM)
Camera Module	Pi Camera Module V2 NoIR
Deployed Model	best320.onnx (YOLOv8n, 320x320)
Inference Runtime	ONNX Runtime (CPU execution provider)
Average Inference Speed	18–25 FPS
Peak Inference Speed	~25 FPS (idle system)
Minimum Inference Speed	~18 FPS (concurrent backend load)
Alert Activation Latency	~150ms (FBS=3 at 20 FPS)
Alert Deactivation Latency	~800ms (CBS=16 at 20 FPS)
Total Validation Inferences	28,159
Total Alert Events Logged	10,641
Offline Operation	Confirmed (no network dependency)

15.2 Model Evolution Performance Comparison

Metric	640x640 ONNX	320x320 RGB Final	IR Raw ONNX	IR CLAHE ONNX	Deployed (best320)
Average FPS (Pi 5)	5.6–5.8	18–25	~20	~20	18–25
Proper Belt (daylight)	Reliable	Reliable	Moderate	Moderate	Reliable
No Belt (daylight)	Reliable	Reliable	Reduced	Moderate	Reliable
Clipped Behind	Not Detected	Improved	Limited	Limited	Improved
Decoy Detection	Inconsistent	Reliable	Limited	Moderate	Reliable
IR (<1 lux) Performance	Poor	Moderate	Moderate	Better	Moderate
Deployment Suitability	Unsuitable	Suitable	Experimental	Experimental	Production

16. Limitations and Known Failure Modes

16.1 Passive Cabin Performance

The RGB-trained production model experiences significant confidence degradation under passive cabin conditions (<1 lux with IR illumination). The spectral distribution of IR imagery differs fundamentally from RGB training data, producing domain shift that manifests as increased false-negative rates. The IR-trained experimental models (best_ir_raw.onnx, best_ir_clahe.onnx) provide partial mitigation but have not been validated to production standard.

16.2 Low-Contrast Scenarios in Dim Light

Black clothing under active cabin conditions (50-100 lux) reduces the contrast between the dark seat belt and the dark garment. The model's ability to localize the no_belt degrades in this configuration. Custom dataset additions targeted this scenario, and detection improved compared to Phase 1, but the edge case of no seatbelt on black shirt in dim light remains the most challenging condition for the deployed system.

16.3 Clipped-Behind Class Sensitivity

The clipped_behind class presents the narrowest visual distinction from proper_belt — the difference is belt routing geometry relative to the torso, not belt presence or absence. Detection is highly sensitive to camera angle and belt visibility. In suboptimal camera positions where the behind-routing is not directly visible, the class degrades to a proper_belt misclassification. Improving this class requires additional annotated data from multiple camera positions and seat configurations.

16.4 Dupatta and Diagonal Fabric Misclassification

Dupattas, scarves, and certain patterned garments that cross the torso at a diagonal angle visually resemble the shoulder-to-hip geometry of a worn seat belt. These objects are occasionally classified as proper_belt rather than decoy. This failure mode is harder to address through data augmentation alone because the visual ambiguity is genuine — from a frontal camera perspective, the temporal or textural differences between a dupatta and a belt may be insufficient for high-confidence discrimination.

16.5 Camera Position Sensitivity

System performance is strongly dependent on camera mounting angle and position. The current test configuration was optimized for a single camera position representing a dashboard-mounted perspective. Significant deviation from this position — for example, door-mounted or overhead cameras — would require revalidation and potentially retraining. This is a fundamental constraint of any fixed-camera vision system.

16.6 Single-Occupant Field of View

The current deployment monitors a single seat position within the camera's field of view. Third-row passenger monitoring, as specified in the qualifying metrics, is architecturally supported through parallel inference streams but requires additional camera hardware and has not been validated.

17. Deployment Guide and Reproducibility

17.1 Environment Setup

The system targets Raspberry Pi OS 64-bit (Bookworm) running on a Raspberry Pi 5. Development and deployment were performed using Python 3.11 inside a dedicated virtual environment (xceed-env). Python 3.11 is the reference interpreter. All software dependencies required for inference, dashboard visualization, backend services, camera interfacing, and GPIO control are documented in the accompanying requirements.txt file. The master.sh script serves as the runtime entry point — these two files have distinct and complementary roles: requirements.txt enables environment portability; master.sh enables operational orchestration. The final deployed inference model was best320.onnx located in ai/best320.onnx. All performance measurements, dashboard demonstrations, database logging results, and final validation videos were generated using this model configuration.

17.2 Launch Procedure

1. Flash Raspberry Pi OS 64-bit to SD card and complete initial setup.
2. Clone the project repository to the Raspberry Pi.
3. Create and activate a Python virtual environment:
   python3 -m venv xceed-env
   source xceed-env/bin/activate
4. Install project dependencies: pip install -r requirements.txt
5. Verify the presence of the deployment model: ai/best320.onnx
6. Execute the master launcher: bash master.sh
7. The launcher automatically:
   • activates the Python environment,
   • starts the FastAPI backend service,
   • starts the Streamlit monitoring dashboard,
   • initializes the alert subsystem,
   • launches the real-time seat-belt detection pipeline.
8. Access the monitoring dashboard through a web browser using: http://192.168.1.20:8501

17.3 Repository and Dependency Structure

The requirements.txt file documents the software dependencies required to reproduce the deployed system, including:

- onnxruntime – execution of the optimized ONNX seat-belt detection model.
- opencv-python – image acquisition, preprocessing, annotation, and video recording.
- numpy – numerical processing and tensor manipulation.
- fastapi and uvicorn – backend REST API services and database communication.
- streamlit – real-time monitoring dashboard.
- picamera2 – camera interface for Raspberry Pi Camera Module 3.
- gpiozero – GPIO-based alert actuation.
- requests – communication between the inference pipeline and backend logging service.
- pandas and pydantic – dashboard and API data handling.

The master.sh script serves as the system orchestration layer. It performs virtual environment activation, launches the FastAPI backend on port 8000, starts the Streamlit dashboard on port 8501, verifies model availability, and executes detect.py as the foreground inference process. Backend and dashboard services are launched as managed background processes with process ID tracking to enable graceful shutdown and cleanup.

18. Future Work

- Production IR Model: Collect a significantly larger IR dataset with diverse subjects and clothing types; train a dedicated IR-optimized model to close the passive-cabin performance gap without relying on the RGB domain-shift workaround.
- Lighting-Adaptive Model Selection: Implement an ambient lux estimator (frame brightness histogram) to dynamically switch between best320.onnx and the IR CLAHE model based on detected lighting conditions.
- Third-Row Monitoring: Extend the system with a second Pi Camera on an additional CSI port or USB camera, feeding a parallel inference stream in detect.py to cover rear-row occupants.
- Low-Contrast Dataset Expansion: Specifically target black belt on black shirt and black belt on dark grey shirt scenarios with a dedicated data collection session, with IR illuminator to also cover dim conditions.
- Dupatta Decoy Improvement: Collect targeted training examples of common Indian garments (dupattas, stoles, printed scarves) at the camera angles they are most likely to present a proper_belt-like profile.
- Multi-Occupant Detection: Extend the model to support bounding-box associations per seat position, enabling simultaneous monitoring of driver and front passenger from a single wide-angle camera.
- Model Quantization: Explore INT8 quantization of best320.onnx using ONNX Runtime quantization tools to further reduce memory footprint and improve FPS headroom for concurrent processes.
- CAN Bus Integration: Interface the alert subsystem with the vehicle CAN bus to expose seat belt status to the central ECU, enabling integration with existing SBR indicator systems.

19. Conclusions

This project successfully developed and deployed a four-class real-time seat belt monitoring system on a Raspberry Pi 5, achieving the fundamental project requirements of offline operation, real-time inference at 18-25 FPS, and class-differentiated alert generation. The engineering journey demonstrated that production deployment of embedded CV systems requires substantially more than model training: dataset composition aligned with deployment conditions proved more impactful than model architecture selection; resolution-throughput tradeoffs required empirical benchmarking rather than theoretical analysis; and hysteresis-based temporal filtering was essential for converting frame-level model outputs into reliable system-level alert behavior.

The three-phase training evolution — public dataset baseline, custom RGB augmentation, and dedicated IR experimentation — provides a documented and reproducible pathway for future improvement. The infrared model experiments established that passive-cabin operation is technically feasible and identified CLAHE preprocessing as a meaningful enhancement for IR-trained model performance. The persistent SQLite violation log, Streamlit monitoring dashboard, and master.sh orchestration script collectively elevate the system from a research prototype to a production-oriented embedded safety system.

Appendix A: Code Repository Structure

The repository evolved from an initial exploratory structure toward a production-oriented deployment layout. The separation between active deployment code (ai/, backend/, frontend/, hardware/) and historical development artifacts (archive/) reflects this maturation.

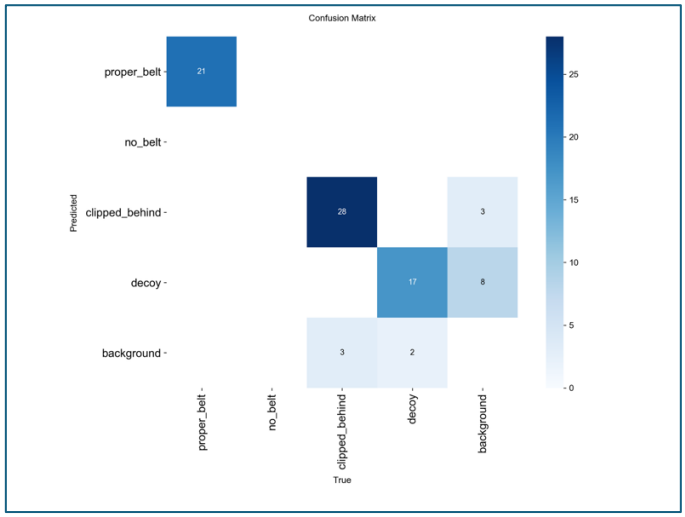
Directory / File	Purpose
ai/	Core AI subsystem. Contains inference engine (detect.py), training script (train_yolo.py), frame extractor (extract_frames.py), and Pi camera capture utility (capture_custom.py), capture live video to check angle of camera and torso framing (live_preview.py), test camera working (camera_test.py)
backend/	FastAPI REST API server (main.py). Handles detection result ingestion, database writes, and dashboard data endpoints.

frontend/	Streamlit monitoring dashboard (dashboard.py). Provides live system status and violation log visualization.
hardware/	GPIO alert controller (gpio_alert.py). Manages LED and buzzer state based on inference decisions.
datasets/	Active training datasets. final_dataset/ (RGB combined), final_dataset_ir_raw/, final_dataset_ir_clahe/. Each contains YOLO-format images/, labels/, and data.yaml.
models/	Deployed model artifacts. best_final.pt (PyTorch checkpoint), yolov8n.pt (base weights). ONNX models (best320.onnx, best_ir_raw.onnx, best_ir_clahe.onnx) referenced from project root.
scripts/	Dataset management utilities: merge_datasets.py, merge_datasets_ir_*.py, create_clahe_dataset.py, validate_dataset.py.
archive/	Historical development artifacts: datasets/ (custom_photos, custom_raw, new_data, public_curated, ir_raw, ir_clahe), training_history/ (per-phase training runs), videos/ (legacy test recordings).
demo_recordings/	Per-scenario validation recordings organized by lighting and clothing condition (01_bright_white, 02_bright_black, 03_dim_white, 04_dim_black, 05_full_demo, 06_offline_proof).
media/	Deployment test videos (deployment_tests/) and model comparison recordings (model_comparison/).
docs/	Architecture diagrams, GPIO wiring schematic, hardware photographs, and dashboard screenshot.
master.sh	System orchestration entry point. Satisfies acceptance criterion for master script without dependency gaps.
README.md	Project setup instructions, hardware requirements, and quick-start guide.

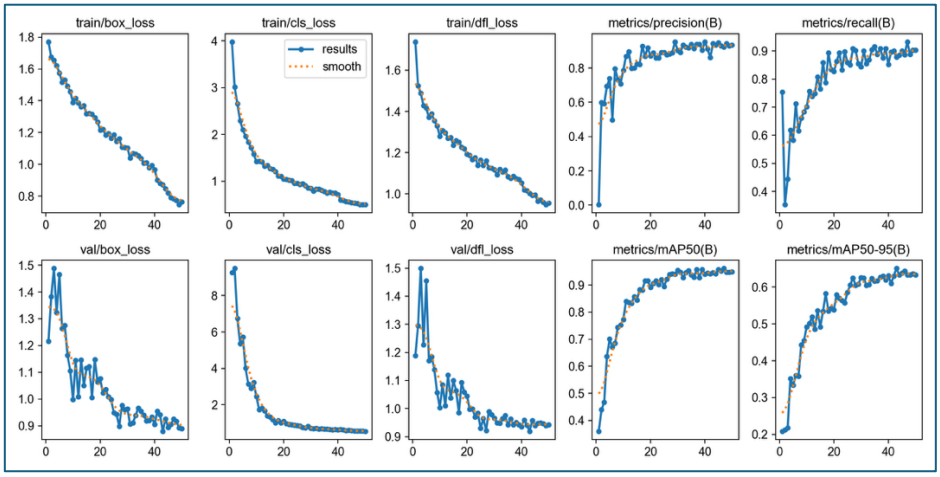
Appendix B: Model Training Artifacts and Validation Evidence

B.1 Training Phase 1 Evidence

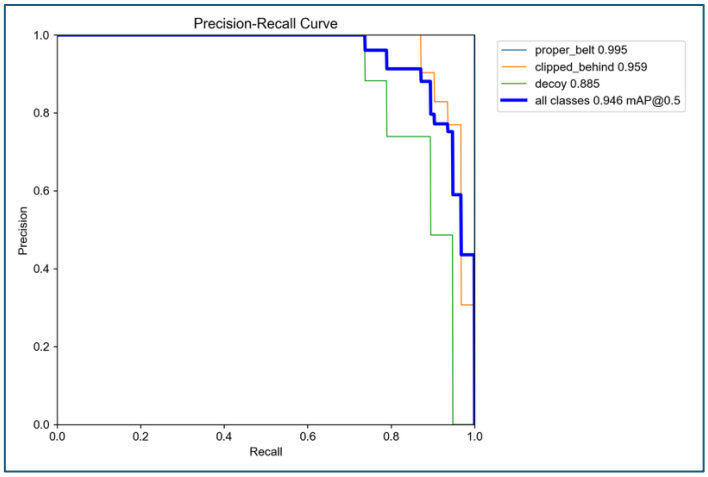
- **confusion_matrix**



- **results**

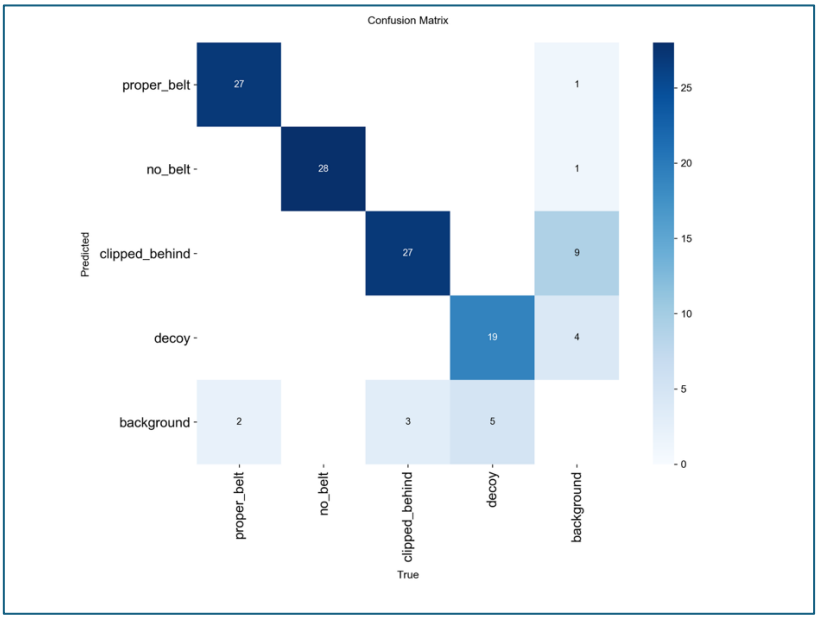


- **BoxPR_curve**

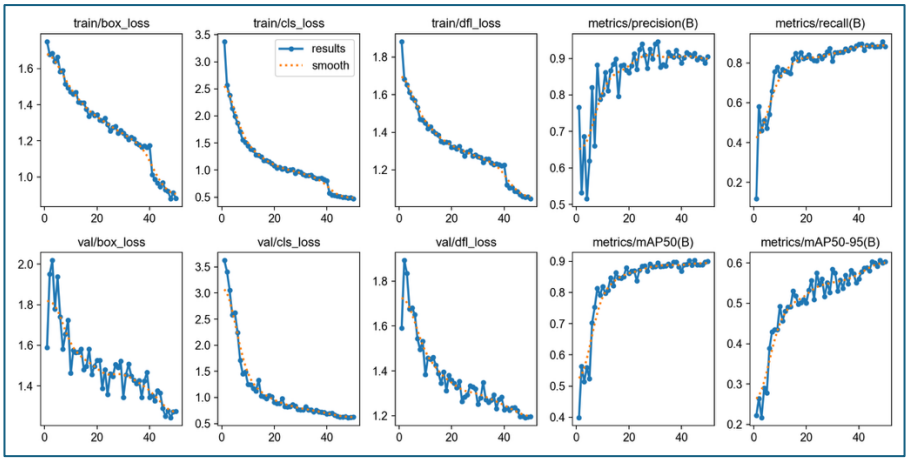


B.2 Training Phase 2 Evidence

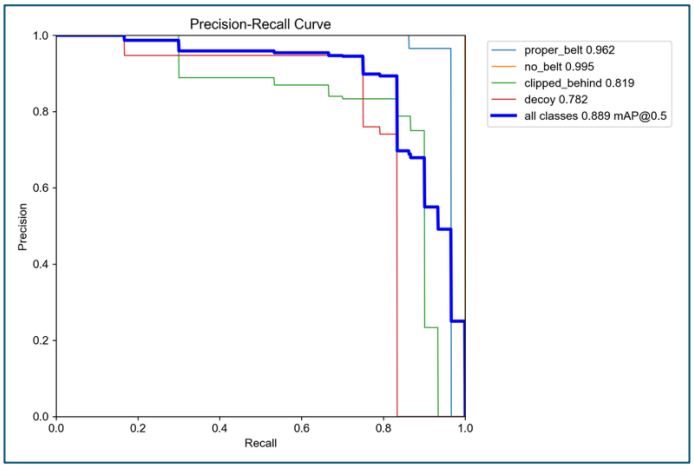
- **confusion_matrix**



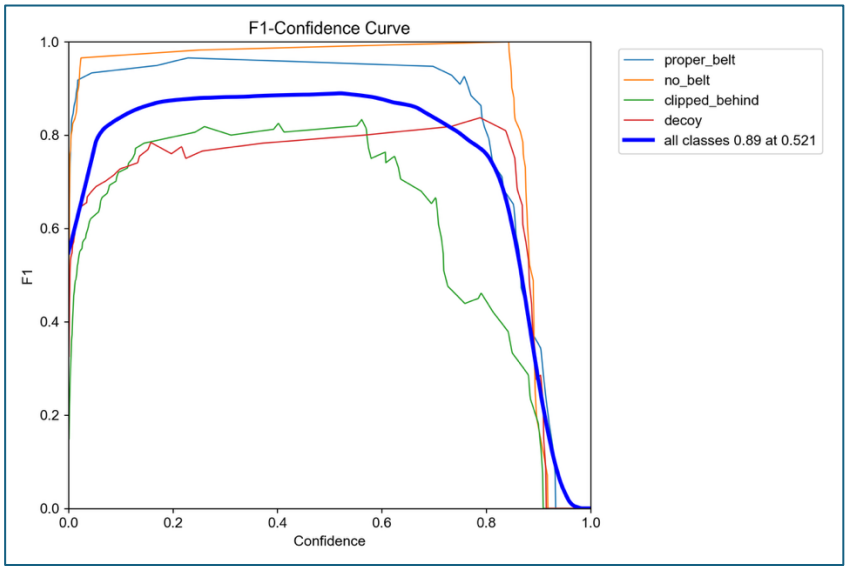
- **results**



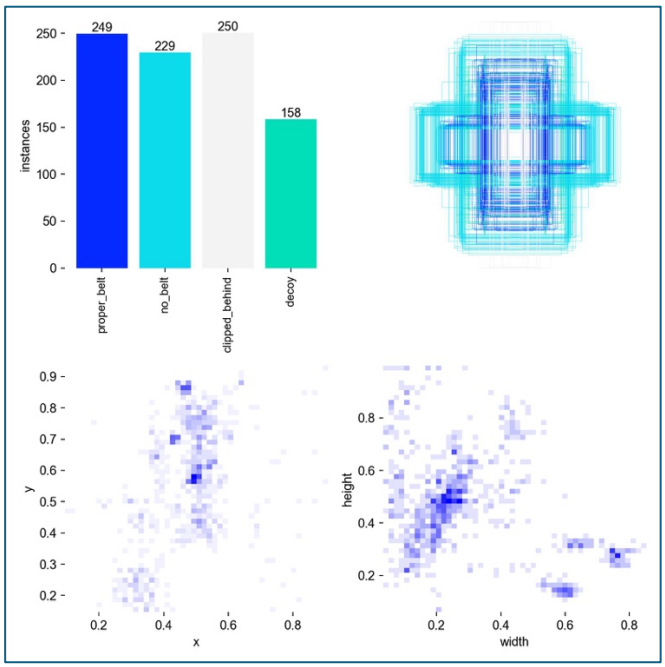
- **BoxPR_curve**



- **BoxF1_curve**



- **labels.png**



References

[1] Ultralytics, "YOLOv8 Documentation," [Online]. Available: https://docs.ultralytics.com

[2] Roboflow, "Seat Belt Detection Dataset V2," Roboflow Universe, [Online]. Available: https://universe.roboflow.com

[3] Microsoft, "ONNX Runtime Documentation," [Online]. Available: https://onnxruntime.ai

[4] Raspberry Pi Foundation, "Raspberry Pi 5 Documentation," [Online]. Available: https://www.raspberrypi.com/documentation

[5] Raspberry Pi Foundation, "Camera Module V2 NoIR," [Online]. Available: https://www.raspberrypi.com/documentation/accessories/camera.html

[6] S. Raka et al., "Real-Time Seat Belt Detection Using YOLOv5 on Edge Devices," IEEE ICCE, 2022.

[7] T. Sebastian et al., "FastAPI: A Modern Framework for Building APIs with Python," [Online]. Available: https://fastapi.tiangolo.com

[8] Streamlit Inc., "Streamlit Documentation," [Online]. Available: https://docs.streamlit.io

[9] OpenCV Developers, "OpenCV 4.x Documentation," [Online]. Available: https://docs.opencv.org

[10] SQLite Consortium, "SQLite Documentation," [Online]. Available: https://www.sqlite.org

[11] K. Zuiderveld, "Contrast Limited Adaptive Histogram Equalization," Graphics Gems IV, Academic Press, 1994.